

Phase 9: Index-Based Sorting — Count Sort, Bin Sort, and Radix Sort

1. Current Progress Before Phase 9

Before Phase 9, you already studied the main comparison-based sorting algorithms.

Phase 1: Sorting Foundations

You learned the basic words used to understand sorting algorithms.

Term	Simple meaning
Sorting	Arranging data in order
Ascending order	Smallest to largest
Descending order	Largest to smallest
Comparison	Checking values to decide their order
Swap	Exchanging two values
Shift	Moving values to make space
Adaptive	Faster when data is already sorted or nearly sorted
Extra memory	Additional space used by an algorithm
Stable	Equal values keep their original relative order
Big O	Describes how algorithm work grows
$O(n)$	Work grows directly with input size
$O(n^2)$	Work grows very quickly, often because of nested loops
$O(n \log n)$	Usually a fast comparison-based sorting class
$O(d \times n)$	Digit-based work, often connected with Radix Sort
Pointer	A variable that stores an address

The most important lesson from Phase 1 was:

Do not judge a sorting algorithm only by asking, “Does it sort?”

A better question is:

How does it sort, how much time does it take, how much memory does it use, and does it preserve duplicate order?

Phase 2: Bubble Sort

Bubble Sort compares neighboring values and swaps them if they are in the wrong order.

Example:

8 5

Since:

8 > 5

Bubble Sort swaps them:

5 8

Important points from Bubble Sort:

- It compares adjacent elements.
 - It uses swaps.
 - One full scan is called a pass.
 - After each pass, one large value is fixed at the end.
 - After k passes, the k largest values are fixed at the end.
 - It can be adaptive if we use a `flag`.
 - It is stable if we use `>` instead of `>=`.
 - Best-case time is $O(n)$ with the adaptive flag.
 - Worst-case time is $O(n^2)$.
 - Space complexity is $O(1)$.
-

Phase 3: Insertion Sort

Insertion Sort builds a sorted left part.

Example:

[5 8] 7 3 2

The left part [5 8] is sorted.

Now we insert 7 into the correct position:

[5 7 8] 3 2

Important points from Insertion Sort:

- It inserts one value into a sorted left part.
 - It starts from index 1 because the first element alone is already sorted.
 - It stores the selected value in `x`.
 - It shifts bigger values to the right.
 - It is naturally adaptive.
 - It is stable if we use `>` instead of `>=`.
 - Best-case time is $O(n)$.
 - Worst-case time is $O(n^2)$.
 - Space complexity is $O(1)$.
-

Phase 4: Selection Sort

Selection Sort selects the smallest value from the unsorted part and puts it at the front.

Example:

8 6 3 2 5 4

The smallest value is 2, so Selection Sort swaps 2 with the first value:

2 6 3 8 5 4

Important points from Selection Sort:

- It selects the minimum value.
- It fills the array from left to right.
- It uses `i`, `j`, and `k`.
- `i` is the position being filled.
- `j` scans the unsorted part.
- `k` stores the index of the smallest value found so far.
- It swaps only after the inner scan finishes.
- It performs fewer swaps than Bubble Sort.
- It is not adaptive.
- It is usually not stable.

- Best-case time is $O(n^2)$.
 - Worst-case time is $O(n^2)$.
 - Space complexity is $O(1)$.
 - After k passes, the k smallest values are fixed at the front.
-

Phase 5 and Phase 6: Quick Sort

Quick Sort introduced partitioning and recursion.

Partitioning means:

Choose one value, called the pivot, and place it in its final sorted position.

Example:

50 70 60 90 40 80 10 20 30 ∞

The pivot is 50.

After partitioning:

40 30 20 10 | 50 | 80 90 60 70

Now 50 is in its final sorted position.

But the left side is not fully sorted:

40 30 20 10

And the right side is not fully sorted:

80 90 60 70

So Quick Sort uses recursion:

1. Partition the array.
2. Sort the left side recursively.
3. Sort the right side recursively.

Simple memory trick:

Partition, sort left, sort right.

Important points from Quick Sort:

- Quick Sort uses divide and conquer.
- It chooses a pivot.
- Partitioning places the pivot in its correct position.
- Recursion sorts the smaller parts.
- Best-case time is $O(n \log n)$.
- Average-case time is $O(n \log n)$.
- Worst-case time is $O(n^2)$.
- First-element pivot Quick Sort can be bad for already sorted data.
- It is usually not stable.
- It mostly sorts inside the original array, but recursion uses stack memory.

Phase 7 and Phase 8: Merge Sort

Merge Sort introduced merging.

Merging means:

Combine two already sorted lists into one bigger sorted list.

Example:

A = [2, 6, 10]
B = [3, 7]

Both lists are already sorted.

After merging:

C = [2, 3, 6, 7, 10]

Important points from Merge Sort:

- Merging works only when the two input parts are already sorted.
- Merging uses i , j , and k .
- i reads the left section.
- j reads the right section.
- k writes into the temporary array.
- A temporary array $B[]$ is used to safely store the merged result.

- Leftover values must be copied after one side finishes.
 - The merged values must be copied from `B[]` back to `A[]`.
 - Iterative Merge Sort uses loops.
 - Recursive Merge Sort uses recursion.
 - Merge Sort has $O(n \log n)$ time complexity.
 - Merge Sort uses extra memory.
 - Merge Sort can be stable when equal values are handled correctly.
-

2. Why Phase 9 Comes After Merge Sort

Until now, most algorithms were **comparison-based**.

A comparison-based sorting algorithm sorts by asking questions like:

```
Is A[i] greater than A[j]?  
Is A[j] greater than A[j + 1]?  
Is this value smaller than the pivot?
```

Examples of comparison-based sorting algorithms:

- Bubble Sort
- Insertion Sort
- Selection Sort
- Quick Sort
- Merge Sort
- Shell Sort

Phase 9 introduces a different group of sorting algorithms.

These algorithms do not mainly sort by comparing pairs of values.

Instead, they use values as:

- indexes
- positions
- buckets
- bins
- digits

This is called **index-based sorting**.

Simple idea:

Earlier algorithms compare values.
Phase 9 algorithms use values, bins, or digits as positions.

3. Main Goal of Phase 9

The main goal of Phase 9 is to understand this idea:

Sorting can sometimes be done faster by using the value itself as an index, bin number, or digit.

Phase 9 covers three algorithms:

Algorithm	Main idea
Count Sort	Count how many times each value appears
Bin Sort	Put values into linked-list bins based on their value
Radix Sort	Sort numbers digit by digit using only 10 bins

Simple memory trick:

Count Sort: count values.
Bin Sort: put values into bins.
Radix Sort: sort digit by digit.

4. What Is Index-Based Sorting?

Index-based sorting means the algorithm does not mainly ask:

Is this value bigger than that value?

Instead, it asks:

Can this value be used as an index?
Can this value be placed into a bin?
Can this digit decide where the number goes?

Example:

Suppose the value `6` appears in the array.

Count Sort may do this:

```
C[6]++;
```

This means:

```
Increase the count at index 6.
```

So the value `6` is used as an index.

5. Comparison-Based vs Index-Based Sorting

Feature	Comparison-based sorting	Index-based sorting
Main method	Compare values directly	Use values, bins, or digits
Example question	Is <code>A[i] > A[j]</code> ?	Can value <code>6</code> go to index <code>6</code> ?
Examples	Bubble, Insertion, Selection, Quick, Merge	Count, Bin, Radix
Speed	Often <code>O(n²)</code> or <code>O(n log n)</code>	Can be near <code>O(n)</code> in suitable cases
Memory	Often lower, depending on algorithm	May need extra arrays or bins
Main limitation	Must compare values	Works best under special conditions

6. Why Index-Based Sorting Can Be Fast

Comparison-based algorithms spend time comparing values.

Example:

```
if (A[j] > A[j + 1])
```

Index-based algorithms can sometimes avoid many of those comparisons.

Example:


```
C[A[i]]++;
```

This directly uses the value as a position.

So if the conditions are suitable, index-based sorting can be very fast.

But there is a trade-off:

Index-based sorting often uses extra memory.

Example:

```
A = [2, 10000]
```

Count Sort may need a counter array from index `0` to index `10000`.

That means `10001` cells for only two input values.

So we must ask:

```
Is the value range small enough?  
Is the extra memory acceptable?  
Are the numbers non-negative?
```

Part A: Count Sort

7. What Is Count Sort?

Count Sort is an index-based sorting algorithm.

It uses an extra array called a **counter array**.

The counter array stores how many times each value appears.

Example:

If the value `6` appears three times, then:

$C[6] = 3$

This means:

The value 6 appears 3 times.

8. Main Idea of Count Sort

Count Sort works like this:

1. Find the maximum value in the array.
2. Create a counter array C of size $\text{max} + 1$.
3. Initialize all positions of C to zero.
4. Scan the original array A .
5. For every value v , do $C[v]++$.
6. Scan the counter array from left to right.
7. Write each index back into A as many times as its count.

Short memory trick:

Find max, count values, write back in order.

9. Why Count Sort Needs the Maximum Value

Count Sort uses values as indexes.

If the largest value is 12, then the counter array needs indexes from 0 to 12.

That means it needs:

$\text{max} + 1 = 12 + 1 = 13$ cells

The indexes are:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12

So before creating the counter array, we must know the largest value.

That is why we use a helper function called `findMax()`.

10. The `findMax()` Helper Function

The job of `findMax()` is simple:

Find the largest value in the array.

Example:

```
A = [6, 3, 9, 6, 12]
```

The largest value is:

```
12
```

So:

```
max = 12
```

C code:

```
int findMax(int A[], int n) {  
    int max = INT32_MIN;  
  
    for (int i = 0; i < n; i++) {  
        if (A[i] > max) {  
            max = A[i];  
        }  
    }  
  
    return max;  
}
```

Explanation

```
int max = INT32_MIN;
```

This starts `max` at a very small integer value.

Then the loop checks every element:

```
if (A[i] > max)
```

If the current array value is bigger than `max`, we update `max`.

At the end, we return the largest value.

11. Count Sort Example

Let us sort this array:

```
A = [6, 3, 9, 6, 12]
```

Final answer should be:

```
[3, 6, 6, 9, 12]
```

12. Step 1: Find the Maximum Value

Array:

```
6 3 9 6 12
```

The maximum value is:

```
12
```

So:

```
max = 12
```

13. Step 2: Create Counter Array

Since `max = 12`, create `C` with indexes from `0` to `12`.

At first, all values are zero:

```
Index: 0 1 2 3 4 5 6 7 8 9 10 11 12
C:      0 0 0 0 0 0 0 0 0 0 0 0 0
```

Each position means:

```
C[0]  = how many 0s
C[1]  = how many 1s
C[2]  = how many 2s
C[3]  = how many 3s
...
C[12] = how many 12s
```

14. Step 3: Count Each Value

Original array:

```
A = [6, 3, 9, 6, 12]
```

Read first value: 6

Do:

```
C[6]++;
```

Now:

```
C[6] = 1
```

Read second value: 3

Do:

```
C[3]++;
```

Now:

```
C[3] = 1
```

Read third value: 9

Do:

```
C[9]++;
```

Now:

```
C[9] = 1
```

Read fourth value: 6

Do:

```
C[6]++;
```

Now:

```
C[6] = 2
```

Read fifth value: 12

Do:

```
C[12]++;
```

Now:

```
C[12] = 1
```

Useful counter values:

```
C[3] = 1  
C[6] = 2
```

```
C[9] = 1  
C[12] = 1
```

15. Step 4: Write Back in Sorted Order

Now scan the counter array from left to right.

Why left to right?

Because index `0` is smaller than index `1`, index `1` is smaller than index `2`, and so on.

So scanning from left to right gives ascending order.

Useful positions:

```
C[3] = 1  
C[6] = 2  
C[9] = 1  
C[12] = 1
```

Write back:

```
Index 3 appears 1 time -> write 3  
Index 6 appears 2 times -> write 6, 6  
Index 9 appears 1 time -> write 9  
Index 12 appears 1 time -> write 12
```

Final sorted array:

```
[3, 6, 6, 9, 12]
```

16. Count Sort Pseudocode

```
CountSort(A, n):  
    max = findMax(A, n)  
    create C[max + 1]  
    set all C values to 0
```

```

for each value in A:
    C[value]++

i = 0
for j from 0 to max:
    while C[j] > 0:
        A[i] = j
        i++
        C[j]--

```

Meaning:

- `j` scans the counter array.
- `i` writes into the original array.
- If `C[j] > 0`, value `j` exists in the original array.
- We write `j` into `A` and decrease its count.

17. Count Sort Code in C

```

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int findMax(int A[], int n) {
    int max = INT32_MIN;

    for (int i = 0; i < n; i++) {
        if (A[i] > max) {
            max = A[i];
        }
    }

    return max;
}

void CountSort(int A[], int n) {
    int max = findMax(A, n);

    int *C = (int *)calloc(max + 1, sizeof(int));

    for (int i = 0; i < n; i++) {
        C[A[i]]++;
    }

    int i = 0;

```



```
int j = 0;

while (j <= max) {
    if (C[j] > 0) {
        A[i++] = j;
        C[j]--;
    } else {
        j++;
    }
}

free(C);
}
```

18. Explanation of Count Sort Code

Header files

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
```

Meaning:

- `stdio.h` is used for input/output, such as `printf()`.
- `stdlib.h` is used for dynamic memory functions like `calloc()` and `free()`.
- `stdint.h` is used for `INT32_MIN`.

Finding the maximum value

```
int max = findMax(A, n);
```

This finds the largest value in the array.

We need this because the counter array size depends on the largest value.

Creating the counter array

```
int *C = (int *)calloc(max + 1, sizeof(int));
```

This creates a dynamic array.

The size is:

```
max + 1
```

Why `max + 1`?

Because if `max = 12`, we need indexes `0` through `12`.

That is 13 positions.

`calloc()` also initializes the array to zero.

That is useful because all counts should start from zero.

Counting values

```
for (int i = 0; i < n; i++) {  
    C[A[i]]++;  
}
```

This scans the original array.

If `A[i] = 6`, then:

```
C[6]++;
```

So the count of value `6` increases.

Writing values back

```
int i = 0;  
int j = 0;  
  
while (j <= max) {  
    if (C[j] > 0) {  
        A[i++] = j;  
        C[j]--;  
    } else {
```

```

        j++;
    }
}

```

Meaning:

- `j` scans the counter array.
- `i` writes into the original array.
- If `C[j] > 0`, then value `j` exists.
- Write `j` into `A[i]`.
- Move `i` forward.
- Decrease `C[j]`.
- If `C[j] == 0`, move to the next counter index.

Freeing memory

```
free(C);
```

This releases the dynamic memory used by the counter array.

In C, if you allocate memory using `calloc()` or `malloc()`, you should free it when it is no longer needed.

19. Important Variables in Count Sort

Variable	Simple meaning
<code>A[]</code>	Original array to sort
<code>n</code>	Number of elements in <code>A</code>
<code>max</code>	Largest value in <code>A</code>
<code>C[]</code>	Counter array
<code>C[A[i]]++</code>	Count one occurrence of value <code>A[i]</code>
<code>i</code>	Position where we write into <code>A</code>
<code>j</code>	Position used to scan <code>C</code>

20. Count Sort Time Complexity

Count Sort does three main tasks:

1. Find the maximum value.
2. Count the values.
3. Scan the counter array and write values back.

If:

```
n = number of input elements
max = largest value
```

Then the time complexity is:

```
O(n + max)
```

Why?

- Finding `max` takes `O(n)`.
- Counting values takes `O(n)`.
- Scanning counter array from `0` to `max` takes `O(max)`.

So total work is:

```
O(n + max)
```

21. Count Sort Space Complexity

Count Sort uses a counter array of size `max + 1`.

So extra space is:

```
O(max)
```

This is why Count Sort can be memory-heavy.

Example:

```
A = [2, 10000]
```

Here:

```
n = 2  
max = 10000
```

Count Sort needs 10001 counter cells for only two values.

This is wasteful.

22. Is Count Sort Stable?

The simple Count Sort version shown here is mainly used for integer values.

It writes values back by scanning the counter array.

For plain integers, stability is usually not visible because equal integers look identical.

But if we were sorting records, this simple version would not preserve full record order unless we modify it.

So for this beginner version:

```
Count Sort is not treated as a record-stable implementation unless modified.
```

Important note:

- Counting values alone is enough for sorting simple integers.
 - Sorting records while preserving equal-order stability requires a more advanced Count Sort version.
-

23. Is Count Sort Adaptive?

Count Sort is not adaptive in the same way as Bubble Sort or Insertion Sort.

Even if the array is already sorted, Count Sort still:

1. Finds the maximum.
2. Creates the counter array.
3. Counts all values.
4. Scans the counter array.

5. Writes values back.

So already sorted data does not make this version stop early.

24. When Count Sort Is Good

Count Sort is good when:

- Values are non-negative integers.
- The maximum value is not too large.
- The range is small compared with the number of elements.
- You want fast sorting and have enough memory.

Example where Count Sort is suitable:

```
A = [3, 1, 4, 2, 3, 0, 2]
```

The largest value is only `4`.

So the counter array is small.

25. When Count Sort Is Not Good

Count Sort is not good when:

- Values are negative, unless the algorithm is modified.
- The maximum value is very large.
- The data is very spread out.
- Memory is limited.

Bad example:

```
A = [2, 1000000]
```

Only two values exist, but the counter array would need `1000001` positions.

Part B: Bin Sort

26. What Is Bin Sort?

Bin Sort is another index-based sorting algorithm.

It is also called Bucket Sort in some contexts.

In this curriculum, Bin Sort uses an array of linked-list bins.

Each bin stores values that match its index.

Example:

If the value is `6`, insert it into bin `6`.

```
Bin[6] -> 6
```

If another `6` appears, insert it again:

```
Bin[6] -> 6 -> 6
```

If a third `6` appears:

```
Bin[6] -> 6 -> 6 -> 6
```

27. Main Idea of Bin Sort

Bin Sort works like this:

1. Find the maximum value.
2. Create bins from index `0` to `max`.
3. Initialize all bins to `NULL`.
4. Scan the original array.
5. Insert each value into its matching bin.
6. Scan bins from left to right.
7. Delete values from bins and write them back into the array.

Short memory trick:

Put values into bins, then empty bins in order.

28. Count Sort vs Bin Sort

Count Sort stores only counts.

Example:

```
C[6] = 3
```

This means value `6` appears 3 times.

Bin Sort stores actual values in linked lists.

Example:

```
Bin[6] -> 6 -> 6 -> 6
```

Both use values as indexes, but the storage method is different.

Feature	Count Sort	Bin Sort
Storage	Counter array	Array of linked-list bins
Example	<code>C[6] = 3</code>	<code>Bin[6] -> 6 -> 6 -> 6</code>
Main operation	Count	Insert and delete nodes
Extra memory	Counter array	Bin array plus linked-list nodes

29. What Is a Bin?

A bin is like a container.

In this algorithm, each bin is a linked list.

Example:

```
Bins[0]  
Bins[1]
```



```
Bins[2]
Bins[3]
...
Bins[max]
```

If value `3` appears, it goes into `Bins[3]`.

If value `6` appears, it goes into `Bins[6]`.

If value `15` appears, it goes into `Bins[15]`.

30. Why Bin Sort Uses Linked Lists

A bin may need to store more than one value.

Example:

```
A = [6, 6, 6]
```

All three values go into bin `6`.

So bin `6` must store multiple nodes:

```
Bins[6] -> 6 -> 6 -> 6
```

A linked list is a simple way to store multiple values inside one bin.

31. Node Structure for Bin Sort

The linked list uses nodes.

Each node stores:

1. The data value.
2. A pointer to the next node.

C structure:

```
struct Node {
    int data;
```

```
struct Node *next;  
};
```

Meaning:

Part	Meaning
data	The value stored in the node
next	Address of the next node

32. FIFO in Bin Sort

FIFO means:

First In, First Out

This means the first value inserted into a bin should be the first value removed from that bin.

Example:

Suppose three values enter bin 6:

6(first), 6(second), 6(third)

With FIFO, they come out in the same order:

6(first), 6(second), 6(third)

This is important because FIFO helps preserve order inside equal values.

FIFO becomes especially important in Radix Sort.

33. Insert Function for Bins

The `Insert()` function inserts a value at the end of a linked list bin.

This gives FIFO behavior.

Code:

```

void Insert(struct Node **bin, int value) {
    struct Node *t = (struct Node *)malloc(sizeof(struct Node));
    t->data = value;
    t->next = NULL;

    if (*bin == NULL) {
        *bin = t;
    } else {
        struct Node *p = *bin;
        while (p->next != NULL) {
            p = p->next;
        }
        p->next = t;
    }
}

```

Explanation

```

struct Node *t = (struct Node *)malloc(sizeof(struct Node));

```

This creates a new node.

```

t->data = value;
t->next = NULL;

```

This stores the value and marks the node as the last node for now.

```

if (*bin == NULL)

```

If the bin is empty, the new node becomes the first node.

```

while (p->next != NULL)

```

If the bin is not empty, move to the last node.

```

p->next = t;

```

Attach the new node at the end.

34. Delete Function for Bins

The `Delete()` function removes the first value from a bin.

Code:

```
int Delete(struct Node **bin) {  
    struct Node *p = *bin;  
    int x = p->data;  
    *bin = (*bin)->next;  
    free(p);  
    return x;  
}
```

Explanation

```
struct Node *p = *bin;
```

`p` points to the first node in the bin.

```
int x = p->data;
```

Save the value before deleting the node.

```
*bin = (*bin)->next;
```

Move the bin head to the next node.

```
free(p);
```

Release the deleted node's memory.

```
return x;
```

Return the removed value.

35. Bin Sort Example

Let us sort:

```
A = [6, 8, 3, 10, 15, 6, 9, 12, 6, 3]
```

Final answer should be:

```
[3, 3, 6, 6, 6, 8, 9, 10, 12, 15]
```

36. Step 1: Find the Maximum Value

Array:

```
6 8 3 10 15 6 9 12 6 3
```

The maximum value is:

```
15
```

So:

```
max = 15
```

37. Step 2: Create Bins

Since `max = 15`, create bins from `0` to `15`.

```
Bins[0]  
Bins[1]  
Bins[2]  
Bins[3]  
Bins[4]  
Bins[5]  
Bins[6]  
Bins[7]
```

```
Bins[8]
Bins[9]
Bins[10]
Bins[11]
Bins[12]
Bins[13]
Bins[14]
Bins[15]
```

At first, all bins are empty:

```
Bins[0] = NULL
Bins[1] = NULL
Bins[2] = NULL
...
Bins[15] = NULL
```

38. Step 3: Insert Values Into Bins

Original array:

```
[6, 8, 3, 10, 15, 6, 9, 12, 6, 3]
```

Insert each value into the bin with the same number.

Insert 6

```
Bins[6] -> 6
```

Insert 8

```
Bins[8] -> 8
```

Insert 3

```
Bins[3] -> 3
```

Insert 10

```
Bins[10] -> 10
```

Insert 15

```
Bins[15] -> 15
```

Insert another 6

```
Bins[6] -> 6 -> 6
```

Insert 9

```
Bins[9] -> 9
```

Insert 12

```
Bins[12] -> 12
```

Insert another 6

```
Bins[6] -> 6 -> 6 -> 6
```

Insert another 3

```
Bins[3] -> 3 -> 3
```

Useful bins after insertion:

```
Bins[3]  -> 3 -> 3  
Bins[6]  -> 6 -> 6 -> 6  
Bins[8]  -> 8  
Bins[9]  -> 9  
Bins[10] -> 10  
Bins[12] -> 12  
Bins[15] -> 15
```

39. Step 4: Empty Bins From Left to Right

Now scan bins from 0 to 15.

Why from left to right?

Because smaller bin indexes represent smaller values.

```
Bins[0]  empty
Bins[1]  empty
Bins[2]  empty
Bins[3]  gives 3, 3
Bins[4]  empty
Bins[5]  empty
Bins[6]  gives 6, 6, 6
Bins[7]  empty
Bins[8]  gives 8
Bins[9]  gives 9
Bins[10] gives 10
Bins[11] empty
Bins[12] gives 12
Bins[13] empty
Bins[14] empty
Bins[15] gives 15
```

Final sorted array:

```
[3, 3, 6, 6, 6, 8, 9, 10, 12, 15]
```

40. Bin Sort Pseudocode

```
BinSort(A, n):
    max = findMax(A, n)
    create Bins[max + 1]
    set every bin to NULL

    for each value in A:
        Insert value into Bins[value]

    i = 0
    j = 0
```



```

while j <= max:
    if Bins[j] is not empty:
        A[i] = Delete from Bins[j]
        i++
    else:
        j++

```

Meaning:

- `j` scans bins from smallest index to largest index.
- `i` writes back into the original array.
- If a bin has values, remove one and write it into `A`.
- If a bin is empty, move to the next bin.

41. Bin Sort Code in C

```

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

struct Node {
    int data;
    struct Node *next;
};

int findMax(int A[], int n) {
    int max = INT32_MIN;
    for (int i = 0; i < n; i++) {
        if (A[i] > max) {
            max = A[i];
        }
    }
    return max;
}

void Insert(struct Node **bin, int value) {
    struct Node *t = (struct Node *)malloc(sizeof(struct Node));
    t->data = value;
    t->next = NULL;

    if (*bin == NULL) {
        *bin = t;
    } else {
        struct Node *p = *bin;
        while (p->next != NULL) {

```

```

        p = p->next;
    }
    p->next = t;
}

int Delete(struct Node **bin) {
    struct Node *p = *bin;
    int x = p->data;
    *bin = (*bin)->next;
    free(p);
    return x;
}

void BinSort(int A[], int n) {
    int max = findMax(A, n);
    struct Node **Bins = (struct Node **)calloc(max + 1, sizeof(struct Node *));

    for (int i = 0; i < n; i++) {
        Insert(&Bins[A[i]], A[i]);
    }

    int i = 0;
    int j = 0;

    while (j <= max) {
        if (Bins[j] != NULL) {
            A[i++] = Delete(&Bins[j]);
        } else {
            j++;
        }
    }

    free(Bins);
}

```

42. Explanation of Bin Sort Code

Creating bins

```
struct Node **Bins = (struct Node **)calloc(max + 1, sizeof(struct Node *));
```

This creates an array of linked-list head pointers.

Each position can point to the first node of a linked list.

Example:

```
Bins[3] -> first node of bin 3  
Bins[6] -> first node of bin 6
```

`calloc()` sets all bin heads to `NULL` at the beginning.

Inserting values

```
for (int i = 0; i < n; i++) {  
    Insert(&Bins[A[i]], A[i]);  
}
```

If `A[i] = 6`, then this inserts `6` into `Bins[6]`.

If `A[i] = 3`, then this inserts `3` into `Bins[3]`.

Emptying bins

```
while (j <= max) {  
    if (Bins[j] != NULL) {  
        A[i++] = Delete(&Bins[j]);  
    } else {  
        j++;  
    }  
}
```

Meaning:

- If bin `j` has values, remove one value and write it into `A`.
 - If bin `j` becomes empty, move to the next bin.
 - Since `j` moves from `0` to `max`, values are written in ascending order.
-

Freeing the bin array

```
free(Bins);
```

This frees the dynamic bin pointer array.

The individual nodes are freed when `Delete()` removes them.

43. Important Variables in Bin Sort

Variable	Simple meaning
<code>A[]</code>	Original array to sort
<code>n</code>	Number of elements
<code>max</code>	Largest value in <code>A</code>
<code>Bins</code>	Array of linked-list bin heads
<code>Bins[A[i]]</code>	Bin that matches the value <code>A[i]</code>
<code>i</code>	Position where we write back into <code>A</code>
<code>j</code>	Scans bins from left to right
<code>p</code>	Pointer used to move through a linked list
<code>t</code>	New node created for insertion
<code>x</code>	Value removed from a bin

44. Bin Sort Time Complexity

Bin Sort scans the original array and inserts values into bins.

Then it scans the bins and deletes values.

If:

`n` = number of input elements
`m` = maximum value range

Then time complexity is often written as:

$O(m + n)$

Why?

- It processes `n` input values.
- It scans bins from `0` to `max`.
- The number of bins depends on the maximum value.

So if the maximum value is small, Bin Sort can be fast.

If the maximum value is huge, scanning many empty bins can waste time.

45. Bin Sort Space Complexity

Bin Sort uses:

1. An array of bin head pointers.
2. Linked-list nodes for the values.

So extra memory is approximately:

$O(\text{max} + n)$

Reason:

- `max + 1` bin positions are needed.
 - `n` linked-list nodes may be created.
-

46. Is Bin Sort Stable?

Bin Sort can be stable if each bin uses FIFO order.

FIFO means:

First In, First Out

If equal values enter the same bin, they leave in the same order.

That preserves the order of equal values.

But if we insert and delete in LIFO order, then equal values may come out reversed.

So the stability of this Bin Sort version depends on FIFO bin handling.

47. Is Bin Sort Adaptive?

Bin Sort is not adaptive in the same way as Bubble Sort or Insertion Sort.

Even if the array is already sorted, Bin Sort still:

1. Finds the maximum.
2. Creates bins.
3. Inserts all values.
4. Scans bins.
5. Writes values back.

So already sorted input does not make it stop early.

48. When Bin Sort Is Good

Bin Sort is good when:

- Values are non-negative integers.
 - The maximum value is not too large.
 - The values fit into a reasonable bin range.
 - You want to group values by index.
 - Memory is available for bins and nodes.
-

49. When Bin Sort Is Not Good

Bin Sort is not good when:

- Values are negative, unless modified.
- The maximum value is very large.
- The data is very spread out.
- Many bins would be empty.
- You want to avoid dynamic memory and linked-list handling.

Bad example:

A = [2, 1000000]

This would need bins from 0 to 1000000 .

That is too much for only two values.

Part C: Radix Sort

50. What Is Radix Sort?

Radix Sort is an index-based sorting algorithm that sorts numbers digit by digit.

It does not need one bin for every possible value.

Instead, for decimal integers, it uses only 10 bins:

```
Bins[0], Bins[1], Bins[2], Bins[3], Bins[4],  
Bins[5], Bins[6], Bins[7], Bins[8], Bins[9]
```

Why only 10?

Because decimal digits are:

```
0 1 2 3 4 5 6 7 8 9
```

So Radix Sort places numbers into bins based on one digit at a time.

51. Why Radix Sort Is Needed

Count Sort and Bin Sort can waste memory if the maximum value is huge.

Example:

```
max = 9999
```

Count Sort may need 10000 counter positions.

Bin Sort may need 10000 bins.

But Radix Sort uses only 10 bins.

Instead of creating one bin for every value, it creates bins for digits:

0 to 9

This reduces the bin range problem.

52. Main Idea of Radix Sort

Radix Sort sorts numbers by digits.

It usually starts from the least significant digit.

Least significant digit means the rightmost digit.

Example:

237

Digits:

2 3 7

- 7 is the ones digit.
- 3 is the tens digit.
- 2 is the hundreds digit.

Radix Sort processes:

ones digit first
tens digit next
hundreds digit next

This is called LSD Radix Sort.

LSD means:

Least Significant Digit

53. Radix Sort Memory Trick

Remember Radix Sort like this:

Sort by ones.
Then sort by tens.
Then sort by hundreds.
Continue until all digits are processed.

Another memory trick:

Digit by digit, from right to left.

54. Why Radix Sort Needs Stable Bins

Radix Sort depends heavily on stability.

When we sort by the next digit, the previous digit order must be preserved inside equal digit groups.

That is why bins must be handled in FIFO order.

FIFO means:

First In, First Out

If we use LIFO, meaning last in first out, the order inside bins can be reversed.

That can destroy the previous digit sorting.

So for Radix Sort:

FIFO is necessary.

55. How Many Passes Does Radix Sort Need?

Radix Sort needs one pass for each digit of the largest number.

Example:

A = [237, 146, 259, 348, 152, 163, 235, 48, 36, 62]

The largest value is:

348

348 has 3 digits.

So Radix Sort needs 3 passes:

Pass 1: ones digit

Pass 2: tens digit

Pass 3: hundreds digit

If the largest number had 4 digits, Radix Sort would need 4 passes.

56. Digit Extraction Formula

To extract a digit, use:

`digit = (A[i] / divisor) % 10;`

The divisor changes each pass.

Pass	Divisor	Digit extracted
Pass 1	1	Ones digit
Pass 2	10	Tens digit
Pass 3	100	Hundreds digit
Pass 4	1000	Thousands digit

57. Digit Formula Example

Suppose:

$A[i] = 237$

Ones digit

$(237 / 1) \% 10$

This becomes:

$237 \% 10 = 7$

So the ones digit is:

7

Tens digit

$(237 / 10) \% 10$

This becomes:

$23 \% 10 = 3$

So the tens digit is:

3

Hundreds digit

$(237 / 100) \% 10$

This becomes:

$2 \% 10 = 2$

So the hundreds digit is:

2

Therefore:

237 -> ones: 7, tens: 3, hundreds: 2

58. Radix Sort Example

Let us sort:

A = [237, 146, 259, 348, 152, 163, 235, 48, 36, 62]

Final answer should be:

[36, 48, 62, 146, 152, 163, 235, 237, 259, 348]

Largest value:

348

Number of digits:

3

So we need 3 passes.

59. Pass 1: Sort by Ones Digit

Look at the ones digit of each number.

237 -> 7
146 -> 6
259 -> 9
348 -> 8

```
152 -> 2
163 -> 3
235 -> 5
48  -> 8
36  -> 6
62  -> 2
```

Now place values into digit bins.

```
Bin[0]: empty
Bin[1]: empty
Bin[2]: 152 -> 62
Bin[3]: 163
Bin[4]: empty
Bin[5]: 235
Bin[6]: 146 -> 36
Bin[7]: 237
Bin[8]: 348 -> 48
Bin[9]: 259
```

Now empty bins from to .

After Pass 1:

```
[152, 62, 163, 235, 146, 36, 237, 348, 48, 259]
```

Now the array is sorted by ones digit.

It is not fully sorted yet.

60. Pass 2: Sort by Tens Digit

Current array:

```
[152, 62, 163, 235, 146, 36, 237, 348, 48, 259]
```

Now look at the tens digit.

```
152 -> 5
62  -> 6
163 -> 6
```

```
235 -> 3
146 -> 4
36  -> 3
237 -> 3
348 -> 4
48  -> 4
259 -> 5
```

Place into bins:

```
Bin[0]: empty
Bin[1]: empty
Bin[2]: empty
Bin[3]: 235 -> 36 -> 237
Bin[4]: 146 -> 348 -> 48
Bin[5]: 152 -> 259
Bin[6]: 62 -> 163
Bin[7]: empty
Bin[8]: empty
Bin[9]: empty
```

Empty bins from to .

After Pass 2:

```
[235, 36, 237, 146, 348, 48, 152, 259, 62, 163]
```

Now the array is sorted by tens digit while preserving the ones-digit order inside equal tens groups.

61. Pass 3: Sort by Hundreds Digit

Current array:

```
[235, 36, 237, 146, 348, 48, 152, 259, 62, 163]
```

Now look at the hundreds digit.

Numbers with fewer than 3 digits have hundreds digit .

```
235 -> 2
36  -> 0
```

```
237 -> 2
146 -> 1
348 -> 3
48 -> 0
152 -> 1
259 -> 2
62 -> 0
163 -> 1
```

Place into bins:

```
Bin[0]: 36 -> 48 -> 62
Bin[1]: 146 -> 152 -> 163
Bin[2]: 235 -> 237 -> 259
Bin[3]: 348
Bin[4]: empty
Bin[5]: empty
Bin[6]: empty
Bin[7]: empty
Bin[8]: empty
Bin[9]: empty
```

Empty bins from to .

Final sorted array:

```
[36, 48, 62, 146, 152, 163, 235, 237, 259, 348]
```

62. Radix Sort Dry Run Summary

Original array:

```
[237, 146, 259, 348, 152, 163, 235, 48, 36, 62]
```

After ones digit pass:

```
[152, 62, 163, 235, 146, 36, 237, 348, 48, 259]
```

After tens digit pass:

```
[235, 36, 237, 146, 348, 48, 152, 259, 62, 163]
```

After hundreds digit pass:

```
[36, 48, 62, 146, 152, 163, 235, 237, 259, 348]
```

63. Radix Sort Pseudocode

```
RadixSort(A, n):  
    max = findMax(A, n)  
  
    for divisor = 1; max / divisor > 0; divisor = divisor * 10:  
        create 10 empty bins  
  
        for each value in A:  
            digit = (value / divisor) % 10  
            Insert value into Bins[digit]  
  
        empty bins from 0 to 9 back into A
```

Meaning:

- `divisor = 1` selects ones digit.
- `divisor = 10` selects tens digit.
- `divisor = 100` selects hundreds digit.
- The loop continues while the maximum value still has digits left.

64. Radix Sort Code in C

This version uses the same linked-list bin functions used by Bin Sort.

```
void RadixSort(int A[], int n) {  
    int max = findMax(A, n);  
    struct Node *Bins[10];  
  
    for (int divisor = 1; max / divisor > 0; divisor *= 10) {  
        for (int i = 0; i < 10; i++) {  
            Bins[i] = NULL;  
        }  
    }
```



```

    for (int i = 0; i < n; i++) {
        int digit = (A[i] / divisor) % 10;
        Insert(&Bins[digit], A[i]);
    }

    int i = 0;
    int j = 0;

    while (j < 10) {
        if (Bins[j] != NULL) {
            A[i++] = Delete(&Bins[j]);
        } else {
            j++;
        }
    }
}
}

```

65. Explanation of Radix Sort Code

Find maximum value

```
int max = findMax(A, n);
```

This finds the largest number.

The largest number tells us how many digit passes are needed.

Example:

```
max = 348
```

Since `348` has 3 digits, Radix Sort needs 3 passes.

Create 10 bins

```
struct Node *Bins[10];
```

Radix Sort only needs 10 bins because decimal digits are `0` to `9`.

Digit pass loop

```
for (int divisor = 1; max / divisor > 0; divisor *= 10)
```

This controls the digit passes.

Divisor	Meaning
1	Ones digit
10	Tens digit
100	Hundreds digit
1000	Thousands digit

The loop continues while:

```
max / divisor > 0
```

This means there are still digits to process.

Initialize bins every pass

```
for (int i = 0; i < 10; i++) {  
    Bins[i] = NULL;  
}
```

Before each digit pass, all bins must be empty.

If we forget this, old values from previous passes may remain.

Extract digit

```
int digit = (A[i] / divisor) % 10;
```

This extracts the current digit.

Example:

If:

```
A[i] = 237
```

and:

```
divisor = 10
```

then:

```
(237 / 10) % 10 = 23 % 10 = 3
```

So the tens digit is **3**.

Insert into digit bin

```
Insert(&Bins[digit], A[i]);
```

If the digit is **3**, the value goes into **Bins[3]**.

Example:

```
237 has tens digit 3  
so 237 goes into Bin[3]
```

Empty bins back into array

```
while (j < 10) {  
    if (Bins[j] != NULL) {  
        A[i++] = Delete(&Bins[j]);  
    } else {  
        j++;  
    }  
}
```

This empties bins from **0** to **9**.

Since lower digit bins come first, this sorts by the current digit.

66. Important Variables in Radix Sort

Variable	Simple meaning
<code>A[]</code>	Original array to sort
<code>n</code>	Number of elements
<code>max</code>	Largest value in the array
<code>Bins[10]</code>	Ten digit bins from <code>0</code> to <code>9</code>
<code>divisor</code>	Selects which digit is being processed
<code>digit</code>	Current digit extracted from the number
<code>i</code>	Scans the array or writes back into <code>A</code>
<code>j</code>	Scans bins from <code>0</code> to <code>9</code>

67. Radix Sort Time Complexity

If:

`n` = number of input elements
`d` = number of digits in the largest value

Then Radix Sort time complexity is:

$O(d \times n)$

Why?

Radix Sort processes all `n` values once for each digit pass.

If the largest number has 3 digits, it makes 3 passes.

So the work is approximately:

$3 \times n$

In general:

$$d \times n$$

So:

$$O(d \times n)$$

68. Radix Sort Space Complexity

Radix Sort uses:

- 10 bins
- linked-list nodes for the values during each pass

Extra memory is usually described as:

$$O(n)$$

The 10 bins are constant because decimal digits are always 0 to 9.

But the nodes can store up to `n` values during a pass.

69. Is Radix Sort Stable?

Radix Sort must be stable to work correctly.

In this linked-list bin version, it is stable if:

Insert happens at the end of the bin.
Delete happens from the front of the bin.

That gives FIFO behavior.

If we use LIFO behavior, Radix Sort may produce wrong results because previous digit order can be destroyed.

70. Is Radix Sort Adaptive?

Radix Sort is not adaptive in the same way as Bubble Sort or Insertion Sort.

Even if the array is already sorted, Radix Sort still processes every digit pass.

Example:

```
[36, 48, 62, 146, 152, 163, 235, 237, 259, 348]
```

This array is already sorted, but Radix Sort still processes ones, tens, and hundreds digits.

71. When Radix Sort Is Good

Radix Sort is good when:

- Values are non-negative integers.
- Values have a limited number of digits.
- You want to avoid a huge `max + 1` counter/bin array.
- You can use stable bins.
- You want near-linear sorting for integer data.

Good example:

```
[237, 146, 259, 348, 152, 163, 235, 48, 36, 62]
```

The largest number has only 3 digits, so only 3 passes are needed.

72. When Radix Sort Is Not Good

Radix Sort is not ideal when:

- Data is not integer-like.
 - Numbers are negative, unless modified.
 - Digit count is very large.
 - Stability cannot be maintained.
 - You want a very simple comparison sort.
-

Part D: Full Phase 9 Program

73. Complete C Program for Count Sort, Bin Sort, and Radix Sort

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

struct Node {
    int data;
    struct Node *next;
};

int findMax(int A[], int n) {
    int max = INT32_MIN;

    for (int i = 0; i < n; i++) {
        if (A[i] > max) {
            max = A[i];
        }
    }

    return max;
}

void CountSort(int A[], int n) {
    int max = findMax(A, n);
    int *C = (int *)calloc(max + 1, sizeof(int));

    for (int i = 0; i < n; i++) {
        C[A[i]]++;
    }

    int i = 0;
    int j = 0;

    while (j <= max) {
        if (C[j] > 0) {
            A[i++] = j;
            C[j]--;
        } else {
            j++;
        }
    }

    free(C);
}
```

```

}

void Insert(struct Node **bin, int value) {
    struct Node *t = (struct Node *)malloc(sizeof(struct Node));
    t->data = value;
    t->next = NULL;

    if (*bin == NULL) {
        *bin = t;
    } else {
        struct Node *p = *bin;
        while (p->next != NULL) {
            p = p->next;
        }
        p->next = t;
    }
}

int Delete(struct Node **bin) {
    struct Node *p = *bin;
    int x = p->data;
    *bin = (*bin)->next;
    free(p);
    return x;
}

void BinSort(int A[], int n) {
    int max = findMax(A, n);
    struct Node **Bins = (struct Node **)calloc(max + 1, sizeof(struct Node *));

    for (int i = 0; i < n; i++) {
        Insert(&Bins[A[i]], A[i]);
    }

    int i = 0;
    int j = 0;

    while (j <= max) {
        if (Bins[j] != NULL) {
            A[i++] = Delete(&Bins[j]);
        } else {
            j++;
        }
    }

    free(Bins);
}

```



```

void RadixSort(int A[], int n) {
    int max = findMax(A, n);
    struct Node *Bins[10];

    for (int divisor = 1; max / divisor > 0; divisor *= 10) {
        for (int i = 0; i < 10; i++) {
            Bins[i] = NULL;
        }

        for (int i = 0; i < n; i++) {
            int digit = (A[i] / divisor) % 10;
            Insert(&Bins[digit], A[i]);
        }

        int i = 0;
        int j = 0;

        while (j < 10) {
            if (Bins[j] != NULL) {
                A[i++] = Delete(&Bins[j]);
            } else {
                j++;
            }
        }
    }
}

void display(int A[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", A[i]);
    }
    printf("\n");
}

int main() {
    int A[] = {6, 3, 9, 6, 12};
    int B[] = {6, 8, 3, 10, 15, 6, 9, 12, 6, 3};
    int R[] = {237, 146, 259, 348, 152, 163, 235, 48, 36, 62};

    CountSort(A, 5);
    printf("Count Sort: ");
    display(A, 5);

    BinSort(B, 10);
    printf("Bin Sort: ");
    display(B, 10);

    RadixSort(R, 10);
}

```

```
    printf("Radix Sort: ");  
    display(R, 10);  
  
    return 0;  
}
```

74. Expected Output

```
Count Sort: 3 6 6 9 12  
Bin Sort: 3 3 6 6 6 8 9 10 12 15  
Radix Sort: 36 48 62 146 152 163 235 237 259 348
```

75. Explanation of the Complete Program

The complete program has these parts:

1. struct Node
2. findMax()
3. CountSort()
4. Insert()
5. Delete()
6. BinSort()
7. RadixSort()
8. display()
9. main()

struct Node

```
struct Node {  
    int data;  
    struct Node *next;  
};
```

This is used for linked-list bins.

Each node stores a value and a pointer to the next node.

findMax()

```
int findMax(int A[], int n)
```

This finds the maximum value.

It is used by Count Sort, Bin Sort, and Radix Sort.

CountSort()

```
void CountSort(int A[], int n)
```

This sorts by counting how many times each value appears.

Insert()

```
void Insert(struct Node **bin, int value)
```

This inserts a value at the end of a linked-list bin.

It gives FIFO behavior.

Delete()

```
int Delete(struct Node **bin)
```

This removes the first value from a linked-list bin.

It also frees the removed node.

BinSort()

```
void BinSort(int A[], int n)
```

This puts each value into its matching bin, then empties bins in order.

RadixSort()

```
void RadixSort(int A[], int n)
```

This sorts values digit by digit using 10 bins.

display()

```
void display(int A[], int n)
```

This prints the array.

main()

The `main()` function creates three arrays:

```
int A[] = {6, 3, 9, 6, 12};  
int B[] = {6, 8, 3, 10, 15, 6, 9, 12, 6, 3};  
int R[] = {237, 146, 259, 348, 152, 163, 235, 48, 36, 62};
```

Then it sorts them using:

```
CountSort(A, 5);  
BinSort(B, 10);  
RadixSort(R, 10);
```

Finally, it prints each sorted array.

Part E: Properties and Comparisons

76. Phase 9 Algorithm Comparison Table

Property	Count Sort	Bin Sort	Radix Sort
Sorting type	Index-based	Index-based	Digit-based index sorting
Main idea	Count values	Put values into bins	Sort digit by digit

Property	Count Sort	Bin Sort	Radix Sort
Uses direct pair comparisons?	No	No	No
Extra memory	Counter array	Bin array + linked-list nodes	10 bins + linked-list nodes
Time complexity	$O(n + \max)$	$O(m + n)$	$O(d \times n)$
Space complexity	$O(\max)$	$O(\max + n)$	Usually $O(n)$ extra
Stable?	Simple integer version: not record-stable unless modified	Stable if FIFO is used	Must be stable using FIFO
Adaptive?	No	No	No
Good for	Small range integers	Reasonable bin range	Multi-digit non-negative integers
Main drawback	Wastes memory if max is huge	Many empty bins if max is huge	Needs stable bin handling

77. What n , $\max$, m , and d Mean

Symbol	Meaning
n	Number of elements in the input array
$\max$	Largest value in the input array
m	Range or maximum bin index used by Bin Sort
d	Number of digits in the largest value

Example:

$A = [237, 146, 259, 348, 152]$

Here:

$n = 5$
 $\max = 348$
 $d = 3$

Because 348 has 3 digits.

78. Memory Trade-Off in Phase 9

Index-based sorting can be fast, but memory matters.

Count Sort example:

A = [2, 10000]

There are only 2 values.

But Count Sort needs indexes:

0 through 10000

That means:

10001 counter cells

This shows the memory trade-off.

So do not think:

$O(n)$ or near $O(n)$ always means best.

A fast algorithm can still be unsuitable if it uses too much memory.

79. Why Radix Sort Reduces the Large-Range Problem

Count Sort and Bin Sort may depend on the maximum value.

If the maximum value is 9999, they may need a large array of size near 10000.

Radix Sort uses only 10 bins for decimal numbers.

Why?

Because each digit is only from 0 to 9.

So Radix Sort avoids creating bins for every possible number.

Instead, it creates bins for digits.

80. Important Difference Between Count Sort, Bin Sort, and Radix Sort

Count Sort

Count Sort asks:

How many times does this exact value appear?

Example:

$C[6] = 3$

Bin Sort

Bin Sort asks:

Which bin should this exact value go into?

Example:

6 goes into Bin[6]

Radix Sort

Radix Sort asks:

What is the current digit of this number?

Example:

```
237 has ones digit 7
so it goes into Bin[7]
```

81. Stability in Phase 9

Stability means equal values keep their original relative order.

This is easy to ignore when sorting plain integers, because equal integers look identical.

But stability matters when sorting records.

Example:

```
6(A), 6(B), 6(C)
```

If the original order is:

```
6(A), 6(B), 6(C)
```

A stable sort keeps the order:

```
6(A), 6(B), 6(C)
```

It should not become:

```
6(C), 6(B), 6(A)
```

For Phase 9:

- Count Sort can be made stable in advanced versions.
 - Bin Sort can be stable if FIFO is used.
 - Radix Sort needs FIFO stability to work correctly.
-

82. Why LIFO Is Bad for Radix Sort

LIFO means:

Last In, First Out

This behaves like a stack.

If values enter a bin like this:

A, B, C

LIFO removes them like this:

C, B, A

That reverses the order.

In Radix Sort, this is dangerous because each pass depends on the order created by the previous pass.

So Radix Sort must use FIFO:

First In, First Out

83. Are Phase 9 Algorithms In-Place?

An in-place algorithm uses little extra memory.

Phase 9 algorithms are generally not in-place in the strict sense.

Why?

- Count Sort uses a counter array.
- Bin Sort uses bins and linked-list nodes.
- Radix Sort uses 10 bins and linked-list nodes.

So they use extra memory.

84. Negative Number Limitation

The versions in this phase assume non-negative integers.

Why?

Because values are used as indexes.

Example:

```
A[i] = 6
```

This is fine:

```
C[6]++;
```

But if:

```
A[i] = -3
```

Then this would be invalid:

```
C[-3]++;
```

Negative array indexes are not allowed in normal C arrays.

So these beginner versions do not directly support negative numbers.

To handle negative numbers, the algorithm must be modified, usually by shifting values using an offset.

Part F: Common Beginner Mistakes

85. Mistake 1: Using Count Sort on Negative Numbers Without Modification

Wrong idea:

```
Use C[A[i]] for negative values.
```

Problem:

```
A[i] = -3
```

Then:

```
C[-3]++;
```

This is invalid.

Correct idea:

Use this beginner version only for non-negative integers, or modify the algorithm with an offset.

86. Mistake 2: Allocating Count Array Based on `n`

Wrong:

```
int C[n];
```

Why wrong?

The counter array needs positions for values, not for the number of elements.

Example:

```
A = [6, 3, 9, 6, 12]
```

Here:

```
n = 5  
max = 12
```

If we create only `C[5]`, we cannot access `C[12]`.

Correct:

```
int *C = (int *)calloc(max + 1, sizeof(int));
```

87. Mistake 3: Allocating Bins Based on `n`

Wrong:

```
struct Node *Bins[n];
```

Why wrong?

Bin indexes are based on values.

If the value `15` appears, we need `Bins[15]`.

So bins must be based on `max + 1`, not `n`.

Correct:

```
struct Node **Bins = (struct Node **)calloc(max + 1, sizeof(struct Node *));
```

88. Mistake 4: Forgetting to Initialize Counters to Zero

If the counter array is not initialized, it may contain garbage values.

Wrong idea:

```
int *C = malloc((max + 1) * sizeof(int));
```

Then using it immediately:

```
C[A[i]]++;
```

Problem:

The starting values may not be zero.

Correct idea:

Use `calloc()`:

```
int *C = (int *)calloc(max + 1, sizeof(int));
```

`calloc()` initializes all positions to zero.

89. Mistake 5: Forgetting to Initialize Bin Heads to `NULL`

Each bin must start empty.

Empty linked-list bin means:

`NULL`

For Radix Sort, bins must be reset to `NULL` at the start of every digit pass.

Correct:

```
for (int i = 0; i < 10; i++) {  
    Bins[i] = NULL;  
}
```

90. Mistake 6: Using LIFO Instead of FIFO in Radix Sort

Wrong behavior:

Last inserted value comes out first.

This can reverse values inside bins.

Correct behavior:

First inserted value comes out first.

That is FIFO.

Radix Sort needs FIFO to preserve digit ordering from previous passes.

91. Mistake 7: Forgetting to Free Dynamic Memory

In C, if memory is allocated dynamically, it should be released.

Examples:

```
free(C);  
free(Bins);
```

Also, linked-list nodes should be freed when deleted:

```
free(p);
```

If you forget this, the program can leak memory.

92. Mistake 8: Thinking Radix Sort Uses `max + 1` Bins

Wrong:

```
Radix Sort needs bins from 0 to max.
```

Correct:

```
Radix Sort needs only 10 bins for decimal digits.
```

The bins are:

```
0, 1, 2, 3, 4, 5, 6, 7, 8, 9
```

93. Mistake 9: Thinking One Radix Pass Fully Sorts the Array

After the ones digit pass, the array is only sorted by ones digit.

Example after Pass 1:

```
[152, 62, 163, 235, 146, 36, 237, 348, 48, 259]
```

This is not fully sorted yet.

Radix Sort must continue with tens digit and hundreds digit.

94. Mistake 10: Confusing `divisor` With `digit`

`divisor` selects the digit position.

`digit` is the extracted digit.

Example:

```
int digit = (A[i] / divisor) % 10;
```

If:

```
A[i] = 237  
divisor = 10
```

Then:

```
digit = 3
```

So:

- `divisor = 10` means we are looking at the tens place.
 - `digit = 3` is the actual digit found.
-

Part G: Key Vocabulary for Phase 9

95. Key Terms

Term	Simple meaning
Index-based sorting	Sorting by using values, bins, or digits as positions

Term	Simple meaning
Count Sort	Sorting by counting occurrences of each value
Counter array	Array that stores how many times each value appears
<code>findMax()</code>	Function that finds the largest value
<code>max</code>	Largest value in the array
<code>max + 1</code>	Required size for Count Sort or Bin Sort indexes from <code>0</code> to <code>max</code>
Bin	A container for values
Bin Sort	Sorting by putting values into bins and emptying bins in order
Bucket	Another name sometimes used for bin
Linked list	Nodes connected using pointers
Node	A structure that stores data and a link to the next node
FIFO	First In, First Out
LIFO	Last In, First Out
Radix Sort	Sorting numbers digit by digit
Digit pass	One round of sorting by one digit position
Ones digit	Rightmost digit
Tens digit	Second digit from the right
Hundreds digit	Third digit from the right
<code>divisor</code>	Used to select ones, tens, hundreds, etc.
<code>digit</code>	The digit extracted from a number
<code>O(n + max)</code>	Count Sort time complexity
<code>O(m + n)</code>	Bin Sort time complexity
<code>O(d × n)</code>	Radix Sort time complexity

Part H: Evaluation Checklist

96. Count Sort Evaluation Checklist

Does it compare values directly?

No.

It uses values as indexes.

Example:

```
C[A[i]]++;
```

Does it use extra memory?

Yes.

It uses a counter array.

Is it adaptive?

No.

Already sorted input does not make it stop early.

Is it stable?

The simple integer version is not treated as record-stable unless modified.

Time complexity

$$O(n + \max)$$

Space complexity

$$O(\max)$$

Best use case

Small non-negative integer range.

97. Bin Sort Evaluation Checklist

Does it compare values directly?

No.

It uses values as bin indexes.

Does it use extra memory?

Yes.

It uses bins and linked-list nodes.

Is it adaptive?

No.

Already sorted input does not make it stop early.

Is it stable?

It can be stable if FIFO is used.

Time complexity

$O(m + n)$

Space complexity

$O(\max + n)$

Best use case

Non-negative integer values with a reasonable maximum value.

98. Radix Sort Evaluation Checklist**Does it compare values directly?**

No.

It sorts by digits.

Does it use extra memory?

Yes.

It uses 10 bins and linked-list nodes.

Is it adaptive?

No.

Already sorted input still goes through digit passes.

Is it stable?

It must be stable to work correctly.

FIFO bins provide stability.

Time complexity

$O(d \times n)$

Space complexity

$O(n)$

Best use case

Multi-digit non-negative integers with a reasonable number of digits.

Part I: Final Summary

99. Phase 9 Big Picture

Phase 9 teaches that sorting does not always need direct comparisons.

Earlier algorithms asked:

Is $A[i]$ bigger than $A[j]$?

Phase 9 algorithms ask:

Can I use this value as an index?
Can I place this value into a bin?
Can I sort this number by its digits?

100. The Three Phase 9 Algorithms

Count Sort

Count Sort counts how many times each value appears.

Memory trick:

Count values, then write them back in order.

Example:

[6, 3, 9, 6, 12] -> [3, 6, 6, 9, 12]

Bin Sort

Bin Sort puts values into bins based on their value.

Memory trick:

Put values into bins, then empty bins from left to right.

Example:

[6, 8, 3, 10, 15, 6, 9, 12, 6, 3]
-> [3, 3, 6, 6, 6, 8, 9, 10, 12, 15]

Radix Sort

Radix Sort sorts numbers digit by digit.

Memory trick:

Ones, tens, hundreds.

Example:

```
[237, 146, 259, 348, 152, 163, 235, 48, 36, 62]  
-> [36, 48, 62, 146, 152, 163, 235, 237, 259, 348]
```

101. Final Beginner Takeaway

Phase 9 is important because it shows a different way of thinking about sorting.

Comparison-based algorithms sort by comparing values.

Index-based algorithms sort by using values, bins, or digits as positions.

Simple final memory trick:

```
Count Sort = count exact values.  
Bin Sort = store exact values in bins.  
Radix Sort = sort by digits using 10 bins.
```

These algorithms can be very fast, but they need the right conditions.

They are best when:

- Values are non-negative integers.
- The value range or digit count is manageable.
- Extra memory is available.
- Stability is handled correctly, especially for Radix Sort.

Next, Phase 10 returns to comparison sorting with Shell Sort, which improves Insertion Sort by using gaps.